Chapitre 5: Héritage

HANENE CHETTAOUI HAMROUNI

MAÎTRE ASSISTANTE EN INFORMATIQUE

Principe de base de l'héritage

L'héritage permet de définir une nouvelle classe à partir d'une classe existante.

Lorsqu'une classe D hérite d'une classe B :

- D possède toutes les caractéristiques de B et aussi, d'autres caractéristiques qui sont spécifiques à D
- D est une spécialisation de B (un cas particulier)
- B est une généralisation de D (cas général)
- D est appelée classe dérivée (fille)
- B est appelée classe de base (mère ou super-classe)

Héritage en JAVA

L'héritage est représenté par la notion d'extension (extends)

[modificateur d'accès] class ClasseDérivée extends ClasseBase

```
class Voiture extends Vehicule
      {
         // Déclaration des attributs et des méthodes
      }
```

Constructeur de la classe dérivée

Le constructeur de la classe mère se charge d'initialiser les attributs hérités, il est désigné par **super()**

L'invocation du constructeur de la classe mère :

super(paramètres du constructeur Mère)

- L'appel du constructeur de la classe mère doit être la première instruction dans le constructeur de la classe dérivée
- A défaut d'appel de super(), JAVA appelle implicitement le constructeur par défaut sans paramètres de la classe mère.

Constructeur de la classe dérivée

```
class Personne {

private String nom;
private int cin;

public Personne(String nom, int cin){
        this.nom=nom;
        this.cin=cin;
}

}
```

```
class Etudiant extends Personne {

private float moyenne;

public Etudiant(String nom, int cin, float
moyenne){
        super(nom,cin);
        this.moyenne=moyenne;
}

}
```

Accès aux membres hérités au sein de la classe dérivée

Un membre hérité (attribut ou méthode) est désigné par **super.membre** (si l'accès est autorisé)

Membre est un attribut:

super.nomAttribut

Membre est une méthode

super.nomMéthode(listeParmètres)

Accès aux membres hérités au sein de la classe dérivée

Accès :	**même classe**	**Classes dérivées dans le même package**	**Classes du même package**	**Classes dérivées dans un autre package**	**Classes des autres packages**
public	**X**	**X**	**X**	**X**	**X**
protected	**X**	**X**	**X**	**X**	
par défaut (friendly)	**X**	**X**	**X**		
private	**X**				

Redéfinition (override) des méthodes

Une méthode héritée peut être redéfinie si sa version initiale n'est pas satisfaisante pour la classe dérivée

La redéfinition consiste à conserver l'entête de la méthode et à proposer un code différent

Lors de la redéfinition d'une méthode, l'appel de l'ancienne version est possible par super.nom_méthode(ListeArguments), et ce dans l'endroit du nouveau code que le programmeur juge adéquat

Redéfinition (override) des méthodes

```
class Personne {

    public void lire() {
            System.out.println("Je sais lire");
    }

}
```

```
class Etudiant extends Personne {

 public void lire() {
            System.out.println("Je suis un étudiant");
            System.out.println("Je sais lire");
    }
}
```

Redéfinition (override) des méthodes

```
class Personne {

    public void lire() {
            System.out.println("Je sais lire");
    }

}
```

```
class Etudiant extends Personne {

 public void lire() {
            System.out.println("Je suis un étudiant");
            super.lire();
    }
}
```

Classe et méthode finales

JAVA permet d'interdire l'héritage d'une classe en la déclarant avec le modificateur final

```
final class NomClasse
```

JAVA permet d'interdire la redéfinition d'une méthode (si elle est héritée) en la déclarant avec le modificateur final

```
final TypeRetour nomMéthode(listeParamètres)
```